Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Сибирский государственный автомобильно-дорожный университет (СибАДИ)»
Институт Информационные системы, экономика и управления
Кафедра Информационная безопасность
Направление 10.03.01 Информационная безопасность

ОТЧЕТ

по лабораторной работе № 5

Тема: «РАЗВЁРТЫВАНИЕ И ИССЛЕДОВАНИЕ ЛОКАЛЬНОЙ БОЛЬШОЙ ЯЗЫКОВОЙ МОДЕЛИ»

по дисциплине «Искусственный интеллект и защита информации»

Выполнил студент группы БИб-23Э1
Филимонов Константин Дмитриевич

Преподаватель:
В.А. Мазур «__» _____2026 г

1. Цель работы

Получить практические навыки развёртывания большой языковой модели на локальном компьютере, подключения к ней Python-клиента через HTTP API, измерения требований к вычислительным ресурсам и производительности, а также оценить преимущества, ограничения и риски информационной безопасности при локальном применении LLM.

Вывод о пригодности модели делается по измерениям, а не по общему впечатлению: время ответа и скорость генерации (задание 4), нагрузка на CPU/RAM/GPU/VRAM (задание 5), повторяемость при разных температурах (задание 6), качество на тестовых и прикладной задаче (задания 2 и индивидуальное). Все результаты получены одним воспроизводимым запуском программы python src/run_all.py, исходный код и сырые данные лежат в репозитории (раздел 5).

2. Конфигурация компьютера

Параметр	Значение
Операционная система	Windows 11 (10.0.26200)
Процессор	Intel Core i5-10300H, 2,5 ГГц
Ядер / потоков	4 / 8
Оперативная память	31,8 ГБ
GPU	NVIDIA GeForce GTX 1650 Ti
Видеопамять	4,0 ГБ
Свободное место на диске C:	53,5 ГБ
Python	3.14.0

Данные собраны автоматически модулем src/sysinfo.py (psutil, nvidia-smi). Объёмы памяти указаны в ГиБ (1024³ байт), как их возвращают инструменты.

3. Среда выполнения и модель

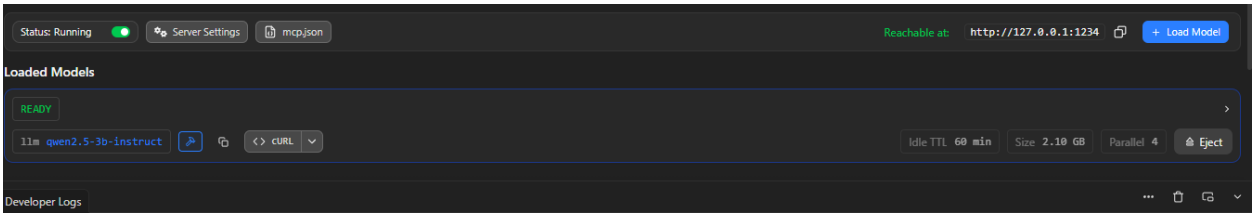
Параметр	Значение
Среда выполнения	LM Studio, локальный сервер (нативный REST API /api/v0 и OpenAI-совместимый /v1)
Версия LM Studio	0.4.23
Движок инференса	llama.cpp-win-x86_64-nvidia-cuda12-avx2, версия 2.50.0
Модель	Qwen2.5-3B-Instruct (издатель Qwen), идентификатор в API: qwen2.5-3b-instruct
Формат / квантование	GGUF, Q4_K_M (4-битное квантование)
Архитектура / размер класса	qwen2, 3B параметров
Размер файла	2,10 ГБ (по окну загрузки LM Studio)
Контекст	загружен 8192 токенов (максимум модели 32768)

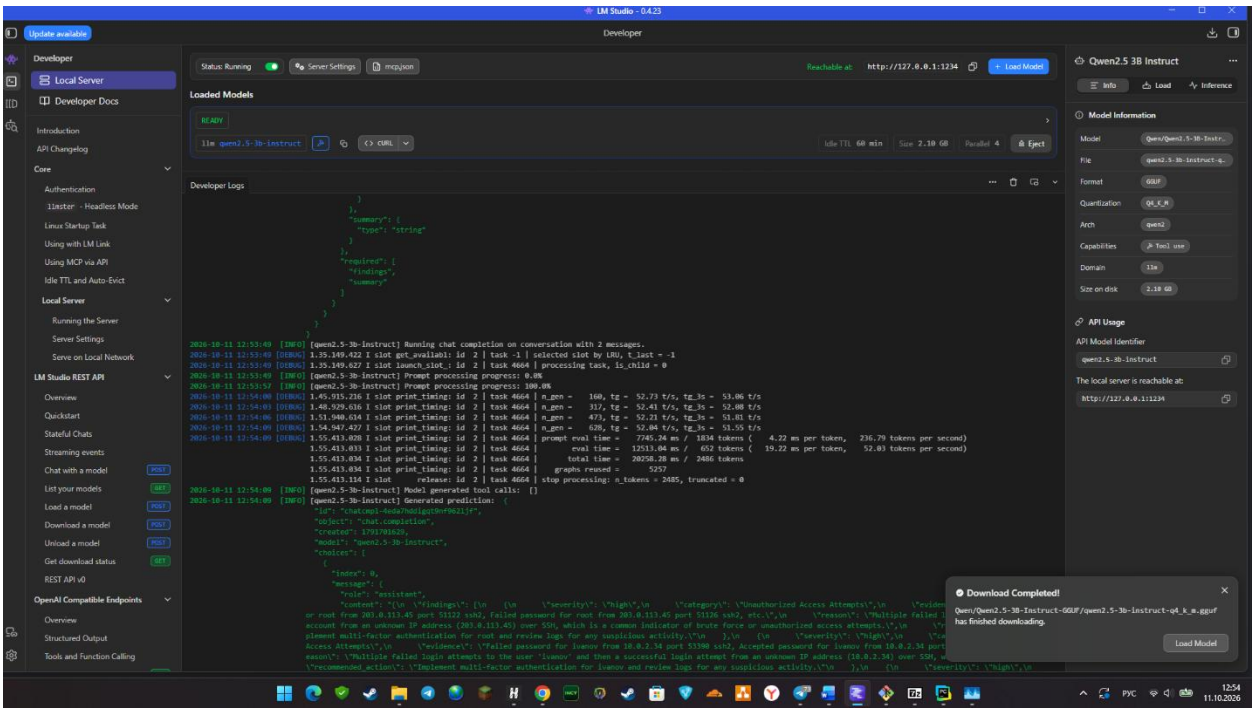
Тип модели	локальная, облачные сервисы не используются
Адрес API	127.0.0.1:1234 (только localhost)

Почему выбрана эта конфигурация. В видеопамяти 4 ГБ должны разместиться веса, кэш контекста и запас под систему. Для модели класса 3B в формате FP16 только веса заняли бы около 6 ГБ ($3 \cdot 10^9$ параметров × 2 байта) и в видеокарту не поместились бы. Квантование Q4_K_M сокращает файл до 2,1 ГБ, и по измерению (раздел 8) модель целиком занимает около 2,3 ГБ VRAM, то есть остаётся запас. Методичка допускает любую компактную инструкционную модель, согласованную с преподавателем; Qwen2.5-3B-Instruct того же класса и семейства, что упомянутые в методичке модели. LM Studio выбрана как среда с готовым локальным API, которую методичка допускает наравне с Ollama.

Проверка источников и сети. Модель и движок скачаны штатными средствами LM Studio из встроенного каталога (в окне загрузки видны llama.cpp 2.50.0 и Qwen2.5 3B Instruct GGUF Q4_K_M). Интернет потребовался только для загрузки; инференс идёт на localhost. Полное отключение сети на время замеров я не проводил, поэтому утверждение «работает офлайн» здесь не проверено.

4. Скриншоты успешного локального запуска






```
  запуск 1/5 (первый, может включать загрузку модели)...
  запуск 2/5...
  запуск 3/5...
  запуск 4/5...
  запуск 5/5...

[4/6] Пять тестовых запросов (задание 2)...
  запрос 1/5: фактологический
  запрос 2/5: суммаризация
  запрос 3/5: классификация
  запрос 4/5: структурированный ответ (JSON)
  запрос 5/5: генерация кода

[5/6] Влияние температуры (задание 6)...
  температура 0.0 (низкая)
  температура 0.8 (средняя)
  температура 1.5 (высокая)

[6/6] Индивидуальное задание: анализ журнала...
```

5. Исходный код клиента

Полный код расположен в репозитории: https://github.com/Konstantin-Filimonov/student-labs/tree/main/lab05. Запуск всей работы одной командой: python src/run_all.py.

Структура: src/client.py — клиент LM Studio (задание 3); sysinfo.py — конфигурация ПК и адрес прослушивания API; monitor.py — мониторинг CPU/RAM/GPU/VRAM каждые 0,5 с (psutil и nvidia-smi); benchmark.py — время и скорость; dialog.py — пять запросов; temperature.py — температура; log_analyzer.py — индивидуальное задание; prompts.py — все промпты и

версия (v1.0); plots.py и report_data.py — графики и сводка. Конфигурация хранится в .env (в Git не попадает), шаблон — .env.example, ключей нет.

Ключевой фрагмент клиента (запрос и расчёт скорости):

```
payload = {"model": self.model, "messages": messages,
           "temperature": temperature, "max_tokens": max_tokens, "stream":
False}
path = "/api/v0/chat/completions" if self.native else "/v1/chat/completions"
t0 = time.perf_counter()
r = self.session.post(self.base_url + path, json=payload,
timeout=self.timeout)
total = time.perf_counter() - t0                  # полное время запроса
data = r.json()
stats = data.get("stats") or {}                    # статистика LM Studio
tps = stats.get("tokens_per_second")
if not tps and completion_tokens:                  # запасной вариант
    tps = completion_tokens / total
```

Нативный эндпоинт выбран потому, что он возвращает время до первого токена и скорость генерации; при его отсутствии клиент переключается на OpenAI-совместимый /v1 и считает скорость по общему времени (источник отмечается в поле stats_source). В этом запуске во всех замерах источником был LM Studio.

6. Результаты пяти тестовых запросов (задание 2)

Запросы отправлялись через API с температурой 0,2 и ограничением 700 токенов. Диалог в окне LM Studio заменён вызовами API: так ответы, время и токены сохраняются в results/dialog.json и results/examples.md. Автоматические проверки в программе формальные, поэтому ниже приведена и ручная оценка.

№	Тип запроса	Время, с	Ток/с	Автопроверка	Ручная оценка
1	Фактологический	5,05	55,6	не пройдена (11 «предложений»)	Частично: суть верна, но 6 предложений вместо 5
2	Суммаризация	4,61	50,7	не пройдена (6 пунктов)	Верно: 3 пункта (в третьем вложенный список)
3	Классификация	2,30	55,5	пройдена: «инцидент»	Верно, объяснение общее
4	JSON	2,29	55,3	пройдена: JSON валиден, 3 поля	Частично: нарушено «только JSON», риск занижен
5	Код Python	7,30	55,2	пройдена: есть def	Частично: решает другую задачу

Запрос 1 (хэширование и шифрование). Модель верно указала необратимость хэша и обратимость шифрования с ключом и назначение обоих механизмов. Недостатки: вводное предложение плюс пять пунктов дают 6 предложений; в текст попали английские слова «one-way» и «reversible»; хэш назван «уникальным», хотя коллизии возможны. Автопроверка показала 11 предложений из-за того, что регулярное выражение разбило текст по номерам «1.», «2.» — это ошибка самой проверки, фактически нарушение формата меньше.

Запрос 2 (суммаризация). Все ключевые тезисы текста сохранены, вымышленных фактов нет, три нумерованных пункта соблюдены. Автопроверка насчитала 6 маркеров, так как посчитала и вложенные элементы списка в пункте 3. Лишь добавлена вводная фраза «Конечно!».

Запрос 3 (классификация). Ответ «Инцидент» допустим для серии неудачных входов с последующим успешным входом с нового IP (допустимым считалось и «подозрительно»). Объяснение общее и не называет прямо перебор паролей.

Запрос 4 (JSON). JSON валиден и содержит три поля, но обёрнут в блок ```json, хотя требовалось вернуть только JSON; наивный json.loads на такой ответ упадёт, в программе ограждение удаляется перед разбором. Уровень риска «medium» занижен: перебор с последующим успехом обычно оценивают как высокий. В пояснении говорится о «новых IP адресах» во множественном числе, хотя в событии один IP.

Запрос 5 (код). Функция синтаксически корректна, но проверяет, **является ли вся строка** адресом IPv4, а не **содержит ли строка** адрес. Ручная проверка на тестовых строках: «192.168.1.1» -> True, «Сервер 192.168.1.1 недоступен» -> False (то есть на исходную постановку функция не отвечает). Кроме того, из-за особенностей int() функция принимает недопустимые строки «1_0.0.0.1», «+1.2.3.4», « 1.2.3.4», «01.02.03.004». Код программой не исполнялся, проверялся вручную отдельно.

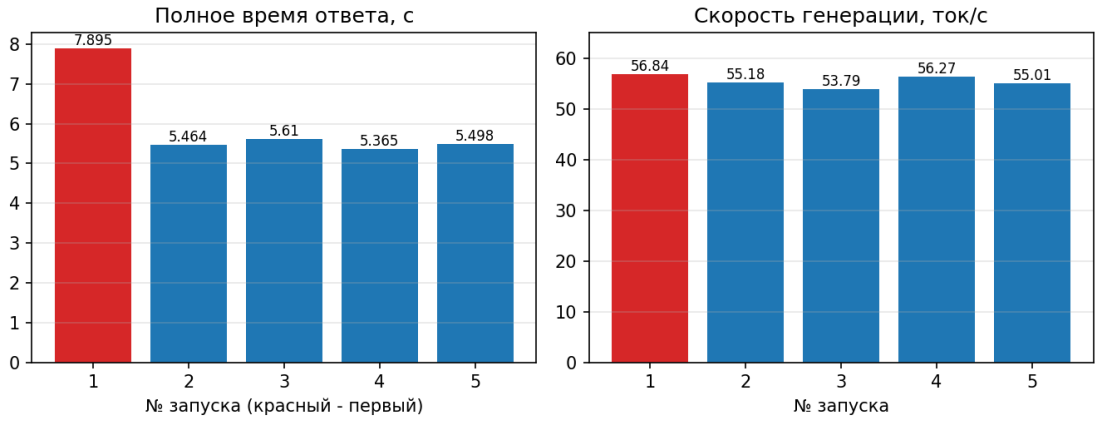
Итог: полностью корректны 2 из 5 ответов, 3 корректны частично, грубо неверных нет. Модель хорошо справляется с пересказом и общими объяснениями, но хуже соблюдает точные ограничения формата и постановки.

7. Время и скорость генерации (задание 4)

Один и тот же запрос «Объясни принцип наименьших привилегий в ИБ.» выполнен 5 раз при температуре 0,2 и ограничении 300 токенов. Перед первым запуском модель была выгружена из памяти (программа проверила состояние not-loaded), поэтому первый запрос включает загрузку.

№	Запуск	Время ответа, с	Время до 1-го токена, с	Выходных токенов	Токенов/с
1	первый	7,895	0,282	300	56,84
2	повторный	5,464	0,020	300	55,18
3	повторный	5,610	0,020	300	53,79
4	повторный	5,365	0,019	300	56,27
5	повторный	5,498	0,020	300	55,01

Среднее время повторных запусков — **5,48 с** (σ = 0,10 с), средняя скорость повторных — **55,06 ток/с**. Первый запуск медленнее в 1,44 раза.



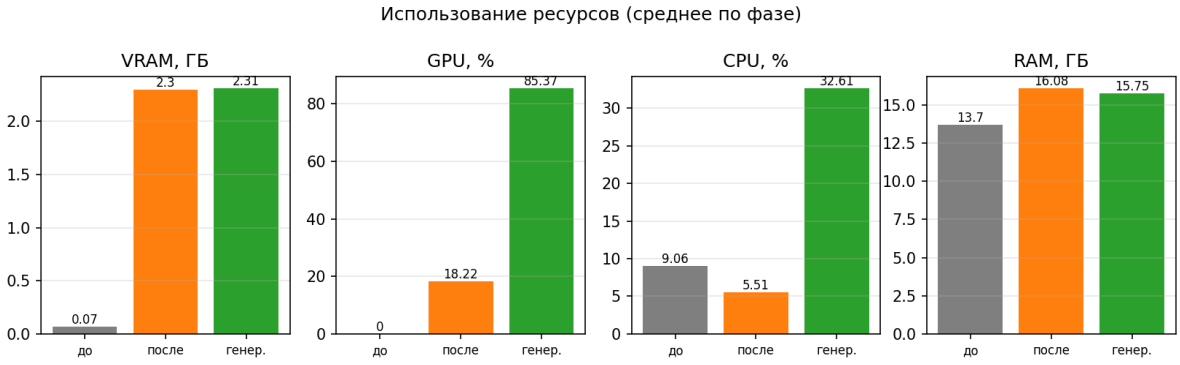
Объяснение разницы. Скорость самой генерации в первом запуске не хуже (56,84 против 55,06 ток/с), поэтому дополнительное время ушло не на генерацию. Если из полного времени вычесть время до первого токена и время генерации (300 токенов / 56,84 ток/с ≈ 5,28 с), остаётся около 2,3 с — это ориентировочная оценка накладных расходов на загрузку весов с диска в память и инициализацию CUDA; напрямую она API не возвращается. Время до первого токена тоже выросло (0,28 с против 0,02 с), но оно составляет лишь

малую часть разницы. Во всех пяти запусках достигнут лимит 300 токенов, то есть ответы обрезаны, но это одинаково для всех запусков и сравнение времени корректно.

8. Использование ресурсов (задание 5)

Метрики снимались фоновым потоком каждые 0,5 с с пометкой фазы: до загрузки (5 с, модель выгружена), после загрузки (5 с простоя сразу после первого запроса), во время генерации (запуски 2–5). CPU и RAM — по всей системе (psutil), GPU и VRAM — через nvidia-smi.

Показатель	До загрузки модели	После загрузки	Во время генерации
RAM, ГБ	13,70 (макс. 13,70)	16,08 (макс. 16,09)	15,75 (макс. 16,11)
Загрузка CPU, %	9,06 (макс. 28,1)	5,51 (макс. 11,1)	32,61 (макс. 78,3)
VRAM, ГБ	0,07 (макс. 0,07)	2,30 (макс. 2,30)	2,31 (макс. 2,33)
Загрузка GPU, %	0,0 (макс. 0,0)	18,22 (макс. 82,0)	85,37 (макс. 92,0)



Столбчатая диаграмма построена по значениям таблицы; временной график с разметкой фаз сохранён программой в results/resources.png.

Что показывают измерения.

- Модель хранится в видеопамяти: VRAM выросла с 0,07 до 2,3 ГБ (+2,2 ГБ). Это согласуется с размером файла 2,1 ГБ и кэшем контекста: по параметрам архитектуры Qwen2.5-3B (36 слоёв, 2 KV-головы, размер головы 128) кэш в формате FP16 занимает около 36 КБ на токен, то есть порядка 0,28 ГБ для загруженных 8192 токенов. Это расчётная оценка, а не измерение. По ней при контексте 32768 токенов кэш вырос бы примерно до 1,1 ГБ. Прямой проверки настройки GPU offload я не делал, но величина VRAM близка к размеру всей модели, что указывает на полную выгрузку слоёв на GPU.

- Во время генерации GPU загружен на 85% (до 92%), то есть скорость ограничена видеокартой. Загрузка CPU тоже растёт (до 78%), но остаётся вспомогательной (токенизация, семплирование, обмен с драйвером).
- Оперативная память выросла на 2,4 ГБ после загрузки, хотя веса находятся в VRAM. Измерение системное, поэтому в него входят и другие программы (исходные 13,7 ГБ заняты не только LM Studio), и кэширование файла модели. Выделить вклад одной LM Studio по этим данным нельзя; снижение с 16,08 до 15,75 ГБ в фазе генерации лежит в пределах колебаний фона.
- Средняя загрузка GPU 18% в фазе «после загрузки», при максимуме 82%, скорее всего, отражает хвост предыдущей генерации в первых отсчётах фазы и интервал усреднения nvidia-smi, а не работу модели в простое. Это предположение: отдельно в простое я загрузку не перепроверял.

9. Влияние параметров генерации (задание 6)

Творческий запрос: «Придумай название и описание из трёх предложений для вымышленного стартапа, который делает продукт в области информационной безопасности». Для каждой температуры сделано по 3 повтора с ограничением 250 токенов. Сходство повторов считалось как среднее попарное сходство Жаккара по множествам слов (1,0 — тексты одинаковы), разнообразие = 1 − сходство. Параметр temperature напрямую поддерживается API LM Studio.

Температура	Сходство / разнообразие	Краткое описание результата	Ошибки и галлюцинации
0,0 (низкая)	0,83 / 0,17	Во всех повторах одно название «SecureSphere» и почти один текст; повторы 2 и 3 совпадают дословно, повтор 1 отличается концовкой.	Фактических ошибок нет. Описание из 4 предложений вместо 3. Ответ не обрезан.
0,8 (средняя)	0,16 / 0,84	Три разных названия (SecureCortex/IntelliGuard/CyberShield, VigilantSec, «Шифровалка»). Вместо одного варианта модель даёт несколько.	Все ответы обрезаны лимитом токенов. В повторе 3 название «КриптоДизайн» не совпадает с

			описанием «Шифровалка…», абсурд «институт» вместо компании.
1,5 (высокая)	0,12 / 0,88	Максимальное разнообразие названий (BitShield, ShieldSecure, VigilantSecure/ShieldGuard), но структура ответа рассыпается.	В повторе 2 модель переключилась на китайский язык посреди слова и сама добавила «перевод» с примечанием; в повторах 1 и 3 бессмысленный заголовок «Название из трёх предложений», нелепые обороты («нежданных хищения», «новые формации атак»); все ответы обрезаны.

Выводы по эксперименту. С ростом температуры повторяемость падает (сходство 0,83 -> 0,16 -> 0,12), а качество при значении 1,5 деградирует до неприемлемого: появляются языковые сбои и бессмыслица. Требование «три предложения» не соблюдено ни при одной температуре. Нулевая температура не дала полностью идентичных ответов: два из трёх совпали, один разошёлся в конце. Вероятной причиной является недетерминизм вычислений на GPU (порядок операций с плавающей запятой), но это предположение, отдельно я его не проверял. Показатель сходства при обрезанных ответах ($T \geq 0,8$) считается по разной длине текста, поэтому он ориентировочный. Практически: для классификации, JSON и разбора журналов нужна температура 0–0,2, для творческих задач разумна 0,7–0,9, значения выше 1,2 для модели этого класса непригодны.

10. Сравнение двух моделей (задание 7)

Вторая модель не запускалась: на видеокарте с 4 ГБ VRAM модель класса 7B в 4-битном виде (файл около 4,7 ГБ по каталогам квантованных версий; на этом ПК не проверялось) целиком не помещается, а запуск с частичной выгрузкой на CPU и большим расходом оперативной памяти методичка не требует. Поэтому задание заменено анализом двух конфигураций по измерениям первой модели и паспортным данным второй. Колонка «Модель B» — оценка, а не измерение.

Критерий	Модель A (измерено)	Модель B (оценка, не измерялась)
Размер/класс	Qwen2.5-3B-Instruct, 3B	Qwen2.5-7B-Instruct, 7B (та же семья, чтобы различалась только величина)
Квантование	Q4_K_M	Q4_K_M
RAM/VRAM	VRAM 2,3 ГБ из 4,0 (запас), RAM +2,4 ГБ	Файл ~4,7 ГБ больше VRAM 4,0 ГБ: только частичная выгрузка слоёв на GPU, остальное в оперативной памяти (31,8 ГБ хватает)
Среднее время	5,48 с на 300 токенов (55 ток/с)	Не измерялось. Генерация упирается в пропускную способность памяти, а вычисления на CPU при частичной выгрузке заметно медленнее GPU, поэтому ожидается существенное замедление
Качество на тесте	См. разделы 6 и 11: ошибки в формате, код не по постановке, пропуск support_tmp	Не измерялось. Ожидается лучшее следование инструкциям; косвенно в ЛР4 модель 7B превосходила модель 3B другой семьи (F1 0,90 против 0,80) на другой задаче, прямого сравнения нет
Итоговое назначение	Быстрый локальный помощник для первичной сортировки и пересказа с проверкой человеком	Задачи, где важнее качество, чем скорость, при условии достаточной памяти или более мощной видеокарты

11. Индивидуальное задание: локальный анализатор журналов событий

Сценарий. Анализ фрагментов журналов событий (из предложенных методичкой направлений). Прототип src/log_analyzer.py принимает файл журнала, передаёт его локальной модели через API и выводит список находок в JSON: severity, category, evidence, reason, recommended_action. Для устойчивого формата использована JSON Schema на стороне сервера (structured output); это сняло проблему нарушения формата, с которой я сталкивался в ЛР4 (там схема была выключена и встречались parse_error).

Здесь ошибок разбора 0. Тест: синтетический журнал из 27 строк (data/sample_auth.txt) с адресами из диапазонов документации и четырьмя заложенными событиями и несколькими штатными (cron-задание, одиночная ошибка пароля штатного пользователя, apt update). Эталон лежит в data/expected_findings.json. Время анализа 20,3 с, выходных токенов 652.

Находка модели	**Что в журнале на самом деле**	**Оценка**
1. Перебор пароля root с 203.0.113.45, high	Шесть неудачных входов и затем успешный вход root (Accepted password)	Найдено частично: успешный вход, то есть факт компрометации, не отмечен. Рекомендация «включить MFA для root» общая
2. Входы пользователя ivanov с «неизвестного» IP 10.0.2.34, high	Одна ошибка пароля и успешный вход; этот IP уже входил по ключу ранее	Ложное срабатывание и фактическая ошибка: «несколько неудачных попыток» и «неизвестный IP»
3. Блокировки UFW с 198.51.100.7, high	10 блокировок на разные порты 21–3389 (сканирование портов)	Найдено, но названо «попытками доступа» к SSH и FTP; рекомендация «MFA для всех» к сканированию не относится
4. www-data читает /etc/shadow, high	Чтение /etc/shadow и загрузка скрипта wget с внешнего адреса	Найдено частично: загрузка скрипта не упомянута, формулировка «доступ к корневому каталогу» неточна
Не найдено	Создание пользователя support_tmp и добавление в группу sudo сразу после входа	Пропуск самого опасного действия (закрепление в системе)

Количественная оценка по эталону (поиск ключевых слов, программа): найдено 3 из 4 заложенных событий (полнота 0,75), 1 ложное срабатывание (ivanov). Ручной разбор строже: из трёх «найденных» событий ни одно не разобрано полностью, а пропущено именно событие, свидетельствующее об успешной компрометации.

Дополнительные наблюдения. Модель ответила по-английски, хотя системный промпт русский; у всех находок одна категория «Unauthorized Access Attempts» и одинаковая severity high, то есть приоритизации нет; рекомендации шаблонные. Эксперимент проведён один раз на одном

небольшом синтетическом журнале, поэтому делать общие выводы о доле ошибок нельзя.

Почему здесь целесообразно локальное размещение. В журналах содержатся внутренние IP-адреса, имена пользователей и хостов, команды администраторов. Локальная модель позволяет разбирать такие данные, не передавая их внешнему сервису, а API слушает только 127.0.0.1. Тарифа за токены нет, объём разбираемых журналов не ограничен лимитами. Минус: качество модели класса 3B недостаточно для самостоятельного решения, она годится как помощник первичной сортировки с обязательной проверкой аналитиком.

12. Меры информационной безопасности

- API-сервер слушает только 127.0.0.1:1234. Адрес определяется программой (psutil), в этом запуске доступ из сети не обнаружен.
- Модель и среда скачаны из встроенного каталога LM Studio; исполняемые файлы неизвестного происхождения не запускались.
- В репозиторий не попадают ключи, пароли и веса модели: .env в .gitignore, остаётся .env.example, весовые файлы (*.gguf) исключены.
- Тестовые данные синтетические, персональных и служебных данных нет.
- Код, сгенерированный моделью, программой не исполняется.
- Локально остаются запросы, ответы и анализируемые журналы. Интернет нужен для загрузки моделей и движков и обновлений LM Studio.

13. Ответы на контрольные вопросы

1. Чем локальный инференс отличается от обращения к облачному API?

При локальном инференсе вычисления выполняются на моём компьютере, а запросы не покидают машину. В облаке запрос уходит провайдеру, есть тариф и зависимость от канала, зато доступны самые мощные

модели. В моей работе все запросы шли на 127.0.0.1, платы за токены не было, но скорость и качество ограничены моей видеокартой.

2. Почему файл модели не равен готовому AI-приложению?

Файл GGUF содержит только веса и метаданные. Для работы нужны движок инференса (llama.cpp), токенизатор, управление памятью и CPU/GPU, а для программного доступа ещё и API-сервер. В моей работе эти роли выполняют llama.cpp CUDA, LM Studio и Python-клиент.

3. Что означает число параметров модели?

Это количество обучаемых весов нейросети (в моём случае около 3 миллиардов). От него зависят объём памяти для хранения и объём вычислений на каждый токен, но не напрямую качество: оно зависит ещё от данных обучения и архитектуры.

4. Для чего применяется квантование?

Для хранения весов в меньшей разрядности (здесь 4 бита вместо 16). Это уменьшает размер модели и объём данных, читаемых из памяти на каждый токен, и ускоряет работу, ценой возможной небольшой потери качества.

5. Почему 4-битная модель может быть удобнее FP16 на персональном компьютере?

Потому что она влезает в доступную память. Модель 3B в FP16 занимает около 6 ГБ и не поместилась бы в мои 4 ГБ видеопамяти, а Q4_K_M занимает 2,1 ГБ и работает целиком на GPU со скоростью около 55 ток/с.

6. Какие ресурсы влияют на скорость локальной генерации?

Пропускная способность памяти и вычислительная мощность GPU, объём VRAM (поместится ли модель целиком), при частичной выгрузке ещё CPU и оперативная память, а также квантование и длина контекста. В моём замере GPU был загружен на 85%, значит, скорость ограничена видеокартой.

7. Почему большой контекст увеличивает расход памяти?

Для каждого токена контекста хранятся ключи и значения внимания (KV-кэш) по всем слоям, поэтому его объём растёт линейно с длиной

контекста. По расчёту для этой модели это около 36 КБ на токен: примерно 0,28 ГБ при 8192 токенах и около 1,1 ГБ при 32768.

8. Что такое токен/с и что эта метрика показывает?

Число сгенерированных токенов в секунду. Метрика показывает скорость генерации (у меня 55,06 ток/с в среднем), но не говорит о качестве ответа и не учитывает время до первого токена и загрузку модели.

9. Почему первый запрос может выполняться медленнее последующих?

При первом запросе веса загружаются с диска в память, инициализируется CUDA и выделяется кэш. У меня первый запрос занял 7,9 с против 5,5 с в среднем, разница около 2,3 с (оценка), а скорость самой генерации не изменилась.

10. Какие данные остаются локально при локальном инференсе и какие действия всё равно могут потребовать Интернет?

Локально остаются запросы, ответы, анализируемые документы и журналы. Интернет нужен для загрузки моделей и движков, обновлений LM Studio и обращения к каталогу моделей. Эти компоненты тоже следует брать из доверенных источников.

11. Какие угрозы возникают, если локальный API открыт на внешний сетевой интерфейс без защиты?

Любой узел сети сможет пользоваться моделью без аутентификации: расходовать вычислительные ресурсы компьютера, получать сведения о нём (например, список моделей), отправлять произвольные запросы. Если к API подключены другие приложения или инструменты, возможно злоупотребление ими через вредоносные промпты. Поэтому API ограничивают localhost или защищают файрволом и аутентификацией на прокси.

12. В каких бизнес-сценариях локальная LLM предпочтительнее облачной?

Когда обрабатываются конфиденциальные данные (персональные данные, журналы ИБ, коммерческая тайна), когда сеть изолирована или есть требования к локализации данных, когда объём запросов велик и важна предсказуемая стоимость, а также когда нужна низкая задержка.

13. В каких случаях облачная модель может быть рациональнее локальной?

Когда нужна максимальная точность и самые большие модели, нагрузка редкая и покупать оборудование нет смысла, нужно быстро масштабироваться, а данные не конфиденциальны или нет ресурсов на эксплуатацию собственной инфраструктуры.

14. Почему меньшая локальная модель не обязательно подходит для всех задач, даже если она быстрее?

Скорость не гарантирует качество. В моих экспериментах модель 3B пропустила создание пользователя support_tmp в журнале, придумала «несколько неудачных входов» у ivanov, написала функцию не под ту постановку и при температуре 1,5 переключилась на китайский язык.

15. Какие данные необходимо измерить, чтобы обоснованно сравнить две локальные модели?

Качество на одном и том же наборе задач (точность или F1, ручная оценка), время полного ответа и до первого токена, токены/с, потребление RAM и VRAM, размер контекста, стабильность формата и повторяемость, параметры запуска (квантование, выгрузка слоёв, температура) и условия (холодный или прогретый старт, число повторов).

14. Вывод о пригодности модели и ограничениях

Qwen2.5-3B-Instruct Q4_K_M успешно развёрнута локально через LM Studio и вызвана из Python по HTTP API. На видеокарте GTX 1650 Ti (4 ГБ) модель целиком помещается в VRAM (2,3 ГБ), генерирует около 55 токенов в секунду и отвечает на запрос из 300 токенов за 5,5 с, а холодный старт добавляет около 2,3 с.

Для каких задач модель пригодна: быстрый локальный пересказ и общие объяснения, первичная сортировка событий и черновой разбор журналов под контролем человека, генерация структурированного JSON при использовании JSON Schema, где данные нельзя отправлять во внешний сервис.

Где её недостаточно: автономные решения в ИБ и любые задачи, где важна точность деталей. Модель не всегда соблюдает точные требования формата (6 предложений вместо 5, ограждение вокруг «только JSON»), решила не ту задачу при написании кода (проверка строки целиком вместо поиска адреса, плюс некорректные принятые значения), занизила риск в JSON, в анализе журнала пропустила создание привилегированной учётной записи и выдумала факт про пользователя ivanov. При температуре 1,5 ответы деградируют (смена языка, бессмыслица), поэтому для практических задач нужны температуры 0–0,2.

Что сделано и что можно улучшить. Для повышения надёжности стоит добавить автоматические тесты для сгенерированного кода, русскоязычные инструкции и примеры в промпт анализатора журналов, повторные прогоны с усреднением, а при наличии ресурсов сравнить с моделью 7B на видеокарте с большей памятью.

Ограничения исследования

- Каждая конфигурация измерялась один раз; для времени и скорости пять запусков, остальное по одному прогону.
- RAM и CPU измерялись по всей системе, поэтому в них входят фоновые программы; настройки GPU offload и версия LM Studio в автоматических данных не фиксировались.
- Автоматические проверки ответов формальные (две из них дали ложное «не пройдено»), поэтому качество оценено вручную одним человеком.
- Индивидуальное задание проверено на одном небольшом синтетическом журнале из 27 строк; сравнение по ключевым словам эвристическое.

- Задание 7 выполнено аналитически, вторая модель не запускалась.